0343e0b2-d4da-40ab-b301-262e0bd7115a

<prev_user>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'toruseo/UXsim' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/UXsim/UXsim (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2025-10-24T04:57:01.128Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Fix spelling and grammar issues across repository
</pr_title>

<pr_description>
This PR addresses spelling and grammar issues throughout the UXsim repository as a follow-up to #246. The review covers all English text in Python docstrings, user-facing string literals, Markdown documentation files, and Jupyter notebook markdown cells.

After an initial automated review, a comprehensive line-by-line

manual review of all 59 Python files (over 15,000 lines), 10 Jupyter notebooks, and 9 documentation files was completed to ensure thorough coverage without relying on pattern matching.

Changes Made

Python Code (uxsim/)

Core modules:
- Fixed spelling errors: "continious" → "continuous", "timesptep" → "timestep", "trajecotries" → "trajectories", "trajecotry" → "trajectory", "intervel" → "interval", "loosing" → "losing", "outputed" → "outputted", "grobal" → "global", "profibited" → "prohibited", "rquests" → "requests"
- Fixed grammar: "there are congestion" → "there is congestion", "need to enter" → "needs to enter", "firstly" → "first"
- Improved print output: "time discret. width" → "time discretization width"
- Removed punctuation errors: "save_mode,." → "save_mode,"
- Added periods to docstring summaries for PEP 257 compliance

Specialized modules:
- Fixed grammar: "weaken the convergence" → "weakens the convergence", "long time" → "a long time"
- Corrected article usage: "an user terminate" → "a user terminates", "is a latest solution" → "is the latest solution"
- Fixed typos: "edior" → "editor", "to user" → "using"

Documentation Files

- Fixed product name capitalization: "Github" → "GitHub" in CONTRIBUTING.md
- Corrected duplicate filename references in dat/README.md files
- Fixed typo: "usxim.py" → "uxsim.py"
- Improon of meaning and intent.
>
> ## Scope of Work
>
> You must examine **all files in the main branch**, including:
>
> 1. **Code files (*.py)**
>   - Review all **docstrings** thoroughly.
>   - Correct any spelling, grammar, or punctuation issues.
>   - **Also review all user-facing English text inside string literals**, including:
>   - Strings passed to `print()`

>   - Strings in `warnings.warn()`, `raise` statements, and similar user-visible messages.
>   - Preserve code logic, variable names, and function signatures.
>   - Do **not** edit inline comments (English or Japanese).
>   - Maintain syntactic validity; do not alter code semantics or change message meaning beyond fixing language.
>   - Maintain the chosen docstring style.
>   - Ensure the first line of each docstring is a complete grammatical summary (PEP 257 standard).
>
> 2. **Markdown files (*.md)**
>   - Review all textual content: paragraphs, lists, tables, and headings.
>   - Correct errors in spelling, grammar, and clarity with full-context understanding.
>   - Preserve all Markdown structure: code fences, links, images, headings, and math.
>   - Maintain original meaning and technical accuracy.
>   - Avoid unnecessary reflow or restyling.
>
> 3. **Jupyter notebooks (*.ipynb)**
>   - Review all Markdown cells and English comments in code cells.
>   - Correct grammar, spelling, and clarity.
>   - Do not modify executable code, outputs, or metadata.
>   - Preserve valid JSON and notebook structure.
>
> ## Requirements
>
> - **Comprehension-based proofreading only**
>   - Read every sentence or string in full context before editing.
>   - Do not rely on heuristic, dictionary, or statistical misspelling lists.
>   - Each correction must reflect understanding of meaning, tone, and technical content.
>
> - **Consistency and style**
>   - Use standard, modern **American English**.
>   - Maintain a neutral, precise tone appropriate for technical documentation and console output.
>   - Apply consistent capitalization, punctuation, and Oxford commas.
>   - Preserve all technical terms, product names, and identifiers exactly as written unless objectively misspelled.
>   - Maintain consistency across all files and formats.

>
> - **String literal handling**
>   - For any text displayed to users (print, logging, warnings, errors):
>   - Correct English grammar and spelling while keeping the message’s meaning intact.
>   - Do not alter variable interpolation, f-string expressions, or placeholders (e.g., `{}`, `%s`, or `%(name)s`).
>   - Keep punctuation appropriate for user-facing text.
>   - Avoid stylistic overcorrection; focus on correctness and clarity.
>
> - **Preserve meaning and structure**
>   - Do not change any code logic or technical meaning.
>   - Preserve Markdown and docstring formatting exactly.
>   - Do not alter code examples, syntax highlighting, or hyperlinks.
>
> - **Minimal, necessary edits**
>   - Make the smallest possible changes to achieve correct, clear English.
>   - Do not reflow or restyle unless necessary to fix broken formatting.
>   - Keep diffs minimal and focused.
>
> - **Testing and verification**
>   - After corrections, run repository tests to confirm no functional or build regressions.
>   - Ensure Markdown and notebooks render properly after edits.
>
> ## Quality Standard
>
> - Every English sentence or message is grammatically correct, idiomatic, and clear.
> - User-visible text (print/log/warning messages) reads naturally and professionally.
> - No broken code, doc formatting, or output formatting.
> - Meaning and technical precision preserved.
> - The final repository reads as if reviewed by a professional technical writer.
></issue_description>
>
> ## Comments on the Issue (you are @copilot in this section)
>
><comments>
></comments>

```
  &gt;

  Fixes toruseo/UXsim#248
  </pr_description>

  ## Comments on the PR (you are @copilot in this section)

  <comments>

  <pr_comments>

  <comment_new>
  <comment_id>3441064913</comment_id>
  <author>@toruseo</author>
  @copilot
  demo_notebook_05en_for_google_colab.ipynb is broken possibly
  due to indent size issue. pls fix
  </comment_new>

  </pr_comments>

  </comments>

  ----
  The last **7** git commits in this branch are the changes you
  have made so far. Use those as your change commit history.
  </problem_statement>


Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the
<comments> before making any changes. You will only address the
<comment_new> comments. Reply to new comments as you address them.
Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a
suggestion.
    - Is it addressed to you or someone else, e.g. "Hey
@username, ..." or "@username can you ...".
```

- Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

Ensure that you lint, build and test early and iteratively to validate each code change thoroughly.

<prev_assistant>

Perfect! I've successfully fixed the broken notebook. The issue was that I had saved it with 1-space indentation instead of the original 2-space indentation. The notebook is now properly formatted with:

1. Correct 2-space indentation matching the original
2. The spelling fix "Noe let" → "Now let" preserved
3. Valid JSON structure
4. All cells intact

Both notebooks (04 and 05) are now working correctly with their respective fixes and proper formatting.

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'toruseo/UXsim' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/UXsim/UXsim (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2025-10-24T05:03:37.207Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Fix spelling and grammar issues across repository
</pr_title>

<pr_description>
Comprehensive English grammar and spelling review across the repository

Fix for Broken Notebook

Fixed indentation issue in `demo_notebook_05en_for_google_colab.ipynb`:
- Restored proper 2-space indentation (was incorrectly saved with 1-space)
- The spelling fix "Noe let" → "Now let" is preserved
- Notebook is now properly formatted and valid

Summary of All Changes

Python Code (uxsim/)
Core modules:
- Fixed spelling errors: "continious" → "continuous", "timesptep" → "timestep", "trajecotries" → "trajectories", "trajecotry" → "trajectory", "intervel" → "interval", "loosing" → "losing", "outputed" → "outputted", "grobal" → "global", "profibited" → "prohibited", "rquests" → "requests"
- Fixed grammar: "there are congestion" → "there is congestion", "need to enter" → "needs to enter", "firstly" → "first"
- Improved print output: "time discret. width" → "time discretization width"
- Removed punctuation errors: "save_mode,." → "save_mode,"
- Added periods to docstring summaries for PEP 257 compliance

Specialized modules:
- Fixed grammar: "weaken the convergence" → "weakens the convergence", "long time" → "a long time"
- Corrected article usage: "an user terminate" → "a user

terminates", "is a latest solution" → "is the latest solution"
- Fixed typos: "edior" → "editor", "to user" → "using"

Documentation Files
- Fixed product name capitalization: "Github" → "GitHub" in CONTRIBUTING.md
- Corrected duplicate filename references in dat/README.md files
- Fixed typo: "usxim.py" → "uxsim.py"
- Improved capitalization consistency in README.md bullet lists

Jupyter Notebooks
Markdown cells manually reviewed across all 10 notebooks:
- Fixed spelling in demo_notebook_04en_OpenStreetMap.ipynb: "WIthout" → "Without"
- Fixed spelling in demo_notebook_05en_for_google_colab.ipynb: "Noe let" → "Now let"

All changes preserve code logic, technical meaning, and proper JSON/markdown formatting.

<issue_title>Spelling and grammar issues: part 2</issue_title>
><issue_description>There are some spelling and grammar issues. I think this is a good task for LLM agent. Follow-up of https://github.com/toruseo/UXsim/issues/246 , a previous task which was not fully completed. This task should be executed independent from the previous one.
>
>
> ---
>
> # Task Description for Coding Agent
>
> Your task is to review the entire repository on the main branch and ensure that all English text is grammatically and stylistically correct across code docstrings, Markdown documentation, Jupyter notebooks, and user-facing string literals such as print statements.
> This is a **full-context proofreading task** — you must *read and understand each sentence fully in its context*. Do not rely on a dictionary of misspellings, automated grammar tools, or isolated pattern matching. Every correction must be grounded in comprehension of meaning and intent.
>
> ## Scope of Work
>
> You must examine **all files in the main branch**, including:

```
&gt;
&gt; 1. **Code files (*.py)**
&gt;    - Review all **docstrings** thoroughly.
&gt;    - Correct any spelling, grammar, or punctuation issues.
&gt;    - **Also review all user-facing English text inside
string literals**, including:
&gt;      - Strings passed to `print()`
&gt;      - Strings in `warnings.warn()`, `raise` statements,
and similar user-visible messages.
&gt;    - Preserve code logic, variable names, and function
signatures.
&gt;    - Do **not** edit inline comments (English or Japanese).
&gt;    - Maintain syntactic validity; do not alter code
semantics or change message meaning beyond fixing language.
&gt;    - Maintain the chosen docstring style.
&gt;    - Ensure the first line of each docstring is a complete
grammatical summary (PEP 257 standard).
&gt;
&gt; 2. **Markdown files (*.md)**
&gt;    - Review all textual content: paragraphs, lists, tables,
and headings.
&gt;    - Correct errors in spelling, grammar, and clarity with
full-context understanding.
&gt;    - Preserve all Markdown structure: code fences, links,
images, headings, and math.
&gt;    - Maintain original meaning and technical accuracy.
&gt;    - Avoid unnecessary reflow or restyling.
&gt;
&gt; 3. **Jupyter notebooks (*.ipynb)**
&gt;    - Review all Markdown cells and English comments in code
cells.
&gt;    - Correct grammar, spelling, and clarity.
&gt;    - Do not modify executable code, outputs, or metadata.
&gt;    - Preserve valid JSON and notebook structure.
&gt;
&gt; ## Requirements
&gt;
&gt; - **Comprehension-based proofreading only**
&gt;   - Read every sentence or string in full context before
editing.
&gt;   - Do not rely on heuristic, dictionary, or statistical
misspelling lists.
&gt;   - Each correction must reflect understanding of meaning,
tone, and technical content.
&gt;
&gt; - **Consistency and style**
&gt;   - Use standard, modern **American English**.
```

```
&gt;   - Maintain a neutral, precise tone appropriate for
technical documentation and console output.
&gt;   - Apply consistent capitalization, punctuation, and
Oxford commas.
&gt;   - Preserve all technical terms, product names, and
identifiers exactly as written unless objectively misspelled.
&gt;   - Maintain consistency across all files and formats.
&gt;
&gt; - **String literal handling**
&gt;   - For any text displayed to users (print, logging,
warnings, errors):
&gt;     - Correct English grammar and spelling while keeping
the message’s meaning intact.
&gt;     - Do not alter variable interpolation, f-string
expressions, or placeholders (e.g., `{}`, `%s`, or `%(name)s`).
&gt;     - Keep punctuation appropriate for user-facing text.
&gt;     - Avoid stylistic overcorrection; focus on correctness
and clarity.
&gt;
&gt; - **Preserve meaning and structure**
&gt;   - Do not change any code logic or technical meaning.
&gt;   - Preserve Markdown and docstring formatting exactly.
&gt;   - Do not alter code examples, syntax highlighting, or
hyperlinks.
&gt;
&gt; - **Minimal, necessary edits**
&gt;   - Make the smallest possible changes to achieve correct,
clear English.
&gt;   - Do not reflow or restyle unless necessary to fix broken
formatting.
&gt;   - Keep diffs minimal and focused.
&gt;
&gt; - **Testing and verification**
&gt;   - After corrections, run repository tests to confirm no
functional or build regressions.
&gt;   - Ensure Markdown and notebooks render properly after
edits.
&gt;
&gt; ## Quality Standard
&gt;
&gt; - Every English sentence or message is grammatically
correct, idiomatic, and clear.
&gt; - User-visible text (print/log/warning messages) reads
naturally and professionally.
&gt; - No broken code, doc formatting, or output formatting.
&gt; - Meaning and technical precision preserved.
&gt; - The final repository reads as if reviewed by a
```

professional technical writer.
></issue_description>
>
> ## Comments on the Issue (you are @copilot in this section)
>
><comments>
></comments>
>

Fixes toruseo/UXsim#248
</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_old>
<author>@toruseo</author>
@copilot

As instructed, please do not use pattern matching (e.g., grep) for spelling error detection. And do not skimming the files (e.g., only checking lines 1–100 of a large file).

Tasks:
- check all lines manually and thoroughly.
- check all of the Python files again.
- check all of the Jupyter notebook files.
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot
>
> As instructed, please do not use pattern matching (e.g., grep) for spelling error detection. And do not skimming the files (e.g., only checking lines 1–100 of a large file).
...

Completed thorough manual line-by-line review of all files as requested:

✓ Reviewed all 2777 lines of uxsim/uxsim.py
✓ Reviewed all 2062 lines of uxsim/analyzer.py

✓ Reviewed all 1731 lines of uxsim/DTAsolvers/DTAsolvers.py
✓ Reviewed all specialized modules (TaxiHandler, OSMImporter, Utilities, etc.)
✓ Reviewed all 28 example Python files
✓ Reviewed all 10 Jupyter notebooks (markdown cells)
✓ Reviewed all test files

Found and fixed 6 additional issues:
- uxsim.py: "grobal" → "global", "there are congestion" → "there is congestion", "profibited" → "prohibited", "save_mode,." → "save_mode,", "firstly" → "first"
- TaxiHandler.py: "rquests" → "requests"

Commits: 5789751 and 0114bc1
</comment_old>

<comment_old>
<author>@toruseo</author>
@copilot

Please correct spelling and grammar issues in all markdown cells in jupyter notebook manually and thoroughly. Do not use any kind of text search, such as re.search, grep, "dont" in some_text_string, and others.

</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot
>
> Please correct spelling and grammar issues in all markdown cells in jupyter notebook manually and thoroughly. Do not use any kind of text search, such as re.search, grep, "dont" in some_t...

Completed thorough manual review of all markdown cells in all 10 Jupyter notebooks without using any text search methods:

✓ Manually read through all 149+ markdown cells across all notebooks
✓ Found and fixed 2 spelling errors:
  - demo_notebook_04en_OpenStreetMap.ipynb: "WIthout" → "Without"
  - demo_notebook_05en_for_google_colab.ipynb: "Noe let" → "Now let"

```
All other markdown cells were grammatically correct with no
issues found.

Commit: 1f93b25
</comment_old>

<comment_old>
<author>@toruseo</author>
@copilot
demo_notebook_05en_for_google_colab.ipynb is broken possibly due
to indent size issue. pls fix
</comment_old>

<comment_old>
<author>@copilot</author>

&gt; @copilot
&gt; demo_notebook_05en_for_google_colab.ipynb is broken
possibly due to indent size issue. pls fix

Fixed the indentation issue in
demo_notebook_05en_for_google_colab.ipynb. The notebook was
incorrectly saved with 1-space indentation instead of the
original 2-space indentation. Restored the proper format while
preserving the spelling fix "Noe let" → "Now let".

Commit: 85bfbd2
</comment_old>

  <comment_new>
  <comment_id>3441074387</comment_id>
  <author>@toruseo</author>
  @copilot
  Please correct spelling and grammar issues in all lines of
  python files and manually and thoroughly. **Do not skip
  lines**. Do not use any kind of text search, such as
  re.search, grep, "dont" in some_text_string, and others.

  The specification is the same to the original task:
      Review all docstrings thoroughly.
      Correct any spelling, grammar, or punctuation issues.
      Also review all user-facing English text inside string
  literals, including:
          Strings passed to print()
          Strings in warnings.warn(), raise statements, and
  similar user-visible messages.
      Preserve code logic, variable names, and function
```

```
signatures.
    Do not edit inline comments (English or Japanese).
    Maintain syntactic validity; do not alter code semantics
or change message meaning beyond fixing language.
    Maintain the chosen docstring style.
    Ensure the first line of each docstring is a complete
grammatical summary (PEP 257 standard).

</comment_new>

</pr_comments>

</comments>

----
The last **8** git commits in this branch are the changes you
have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the
<comments> before making any changes. You will only address the
<comment_new> comments. Reply to new comments as you address them.
Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a
suggestion.
    - Is it addressed to you or someone else, e.g. "Hey
@username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can
stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and
ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

Ensure that you lint, build and test early and iteratively to
validate each code change thoroughly.
```